

Aula 06 - Exercícios de JavaScript

Descrição: Atividade prática desenvolvida para consolidar o aprendizado dos conceitos fundamentais da linguagem JavaScript. O objetivo principal da aula não é apenas fazer o aluno escrever ou memorizar códigos, mas desenvolver a sua autonomia, capacidade de análise crítica e capacidade de explicação da lógica de programação.

Regras de Execução e Entrega dos Exercícios

Nesta atividade, o uso de Inteligências Artificiais (Gemini, ChatGPT, Copilot) é **permitido e encorajado** para auxiliar na geração dos códigos. No entanto, o código puro não será o foco da avaliação. Para que um exercício seja considerado "Correto", ele deve cumprir obrigatoriamente as 3 etapas abaixo:

1. O Código Funcional (A base)

O código deve resolver o problema proposto no enunciado. Se você usou uma IA para gerar a resposta, você é responsável por garantir que o código funciona e atende exatamente ao que foi pedido.

2. Apropriação por Comentários (Obrigatório)

Você deve inserir comentários no código explicando **como** a solução funciona. Códigos gerados por IA sem comentários ou com comentários genéricos serão desconsiderados.

- **Módulos 1 a 3:** Comente o que cada variável guarda e qual é a lógica da condição (if/else) ou repetição (for/while).
- **Módulos 4 e 5:** Explique o que entra como parâmetro na função, o que ela retorna, e como métodos como `.map()` ou `.filter()` estão transformando os dados.

3. Prova de Execução (Casos de Teste)

A IA gera a função, mas você deve provar que a testou. Para todo exercício que cria uma lógica ou função, você deve adicionar no mínimo **3 chamadas (testes) no console** demonstrando resultados diferentes.

- **Exemplo prático** (Exercício 12 - IMC): Além da função gerada, seu arquivo deve conter:

JavaScript

```
console.log(calcularIMC(70, 1.75)); // Teste 1: Peso normal
console.log(calcularIMC(50, 1.75)); // Teste 2: Abaixo do peso
console.log(calcularIMC(100, 1.60)); // Teste 3: Obesidade
```

Guia Prático: Resolvendo a Lista de JavaScript no VS Code

Este guia vai te ajudar a configurar seu ambiente e executar os códigos dos **Módulos 1 ao 5** (Fundamentos, Condicionais, Laços, Funções e Callbacks) diretamente no seu computador, utilizando uma abordagem profissional de desenvolvimento.

Como nestes módulos nós não usaremos navegadores de internet (HTML/CSS), todas as saídas de dados serão vistas no **Terminal** usando o `console.log()`.

Passo 1: Preparação do Ambiente (O que você precisa ter instalado)

Antes de começar, você precisa de duas ferramentas instaladas no seu computador:

1. **Visual Studio Code (VS Code).**
2. **Node.js.**

Passo 2: Criando a Estrutura do Projeto

Um bom desenvolvedor é organizado. Não crie tudo em um arquivo só!

1. Crie uma pasta no seu computador chamada Aula06-Exercicios-JS.
2. Abra o VS Code. Vá em **File (Arquivo) > Open Folder (Abrir Pasta)** e selecione a pasta que você acabou de criar.
3. Dentro do VS Code, crie 5 subpastas, uma para cada módulo da lista:
 - o modulo-1-fundamentos
 - o modulo-2-condicionais
 - o modulo-3-lacos
 - o modulo-4-funcoes
 - o modulo-5-callbacks
4. Dentro de cada pasta, você criará um arquivo JavaScript para cada exercício. Por exemplo, na pasta do módulo 1, crie ex01-variaveis.js, ex02-constante.js, etc.

Passo 3: Escrevendo o Código e Aplicando as Regras de Entrega

Lembre-se da nossa regra de uso da Inteligência Artificial: **Você pode gerar o código, mas precisa provar que o entende!**

Vamos pegar como exemplo o **Exercício 15** do Módulo 2 (verificar se um número é par ou ímpar usando módulo %). Abra o arquivo `/modulo-2-condicionais/ex15-par-impair.js` e siga a estrutura obrigatória:

JavaScript

// Passo 1: A Função (Criada por você ou pela IA)

```
function verificarParOuImpar(numero) {
```

// Passo 2: Apropriação por Comentário (OBRIGATÓRIO)

// O operador % (módulo) pega o resto da divisão. Se o resto da divisão por 2 for zero, é par.

```
if (numero % 2 === 0) {
```

```
    return "O número " + numero + " é Par";
```

```
} else {
```

```
    return "O número " + numero + " é Ímpar";
```

```
}
```

```
}
```

// Passo 3: Prova de Execução (No mínimo 3 testes usando console.log)

```
console.log(verificarParOuImpar(10)); // Teste 1: Número par
```

```
console.log(verificarParOuImpar(7)); // Teste 2: Número ímpar
```

```
console.log(verificarParOuImpar(0)); // Teste 3: O zero (deve retornar par)
```

Passo 4: Como Executar o Código no Terminal do VS Code

Esta é a parte mais importante. Como você vai ver se o código funcionou?

1. No VS Code, vá no menu superior em **Terminal > New Terminal (Novo Terminal)**. O atalho é Ctrl + `.
2. Um painel vai se abrir na parte de baixo da tela.
3. Você precisa "navegar" até a pasta onde seu arquivo está. No terminal, digite:
`cd modulo-2-condicionais` (e aperte Enter).
4. Agora, mande o Node.js executar o seu arquivo digitando:
`node ex15-par-impair.js` (e aperte Enter).
5. **A Mágica Acontece:** Você verá o resultado dos seus `console.log()` impressos diretamente na tela preta do terminal!

Passo 5: Dicas de Ouro para os Módulos 4 e 5

À medida que você avança para os **Módulos 4 (Arrow Functions, Spread/Rest)** e **Módulo 5 (Funções de Alta Ordem como map, filter e reduce)**, os códigos ficam mais enxutos, mas a lógica fica mais complexa.

- No Exercício 38 (usar `.filter()` para retornar números maiores que 3): Crie o array de teste antes de chamar a função (ex: `let numeros = [1, 2, 3, 4, 5];`) e imprima o array antigo e o array novo gerado pelo `.filter()` para comprovar a transformação.
- No Exercício 40 (Simular atraso com `setTimeout`): Ao rodar o comando `node seu_arquivo.js`, você perceberá que o terminal vai "pausar" por alguns segundos antes de imprimir a resposta final e liberar o cursor. Isso é o comportamento assíncrono acontecendo na prática no seu computador!

Resumo do Fluxo de Trabalho

1. Criar o arquivo `.js`
2. Escrever a lógica
3. Comentar como funciona
4. Chamar a função em `console.log()`
5. Rodar no terminal com `node nome_do_arquivo.js`
6. Validar o resultado
7. Ir para o próximo!